Making the upload fast, and uploading each thing once

19 Oct 2026 · proposal, nothing changed yet

> *## U1 is retracted — measured 2 Oct, 15 clips I ranked "skip the proxy, the source is already proxy-grade" first, on an estimate that the clips come out of* `clips:day` *at 2–3 Mbps.* ***They come out at a mean of 6.10 Mbps****, with four of fifteen pinned against the 8 Mbps cap. | | |---|---| | 15 clips, 22.4 min |* ***1027 MB*** *| | range | 3.38 – 8.02 Mbps | | mean |* ***6.10 Mbps*** *| | proxy at 1.2 Mbps would be |* ***202 MB — 20% of the clips, not the ~48% I assumed*** *| So the proxy is a genuine* ***5× reduction****, not the near-no-op I described. It is earning its place and should not be skipped. The guard I proposed — height ≤ 720* ***and*** *bitrate ≤ 2 Mbps — is still correct, and would simply* ***never fire on this workload****: the height test passes, the bitrate test fails on every clip. Cheap insurance for a future re-encoded source, worth nothing today, and certainly not item 1. §1 and §2 below are wrong where they reason from the low estimate. §9 has the measurement and the re-ranked order. U2–U10 are unaffected — none of them depended on the bitrate. The measurement also moves the real question into view, the one §6 of the earlier doc raised and I declined to push:* ***the original is 83% of everything we upload.*** *That, not the proxy, is the byte budget.*

Focused on the upload path only. Day-open and boot are in `upload-performance-proposal-2026-10.md` and are not repeated here.

Three questions, in the order they matter:

1. **How many bytes do we send?** — §1–§2. This is where the big factor is, and it is not where I expected.
2. **Do we send anything twice?** — §3–§4. Yes, in three distinct ways.
3. **How cheaply can we tell what is already there?** — §5. Currently O(N²) directory listings against a flat `size` comparison with no content identity.

---

1. The byte budget, which reframes everything

The code is written for a world where "the original" is a multi-GB 4K camera file. `useCloudSync.js:373` says so: "*originals are multi-GB, so the user triggers this only when on fast wifi.*"

For the main workflow that is no longer true. `clips:day` → `compress-videos.sh:159` already encodes every clip to:

```
-vf scale=-2:720  -c:v libx264 -crf 26  -maxrate 8M  -movflags +faststart
```

So what lands in `~/clips`, and what the Upload tab's watcher imports, is **already 720p H.264 with faststart** — typically 2–3 Mbps for sailing footage, roughly 25–35 MB for a 90 s race clip.

SSA then treats that file as "the original" and generates a **proxy** from it: `video-proxy.ts:50` targets **720p at 1.2 Mbps**. Same resolution. A pure bitrate reduction of an already-compressed 720p file, and a generation of quality lost on the way.

	clips:day output ("original")	SSA proxy
Resolution	720p	**720p**
Bitrate	~2–3 Mbps (CRF 26, cap 8M)	1.2 Mbps
90 s clip	~30 MB	~14 MB

Both are uploaded. So on a 20-clip day we send ~600 MB of originals **plus ~270 MB of proxies that are 720p re-encodes of 720p sources** — about a third of everything that crosses the uplink, for a file whose only advantage is being half the bitrate of something already small.

`generateProxy()` never looks at the source. There is no height check, no bitrate check — `useCloudSync.js:366` queues every clip with `!v.hasProxy` and `video-proxy.ts:240` unconditionally scales to 720 (a no-op here) and re-encodes.

The proxy earns its place on the drone-card path — 4K → 720p is a genuine 20× reduction, and a phone on marina 4G should not be pulling 4K. It earns nothing on the `clips:day` path, which per CLAUDE.md is *the* path.

U1 — Skip the proxy when the source already satisfies it

Both inputs are already in hand at import: `probeVideo` returns `width/height` and `duration` (`videoProbe.js:220`), and `file.size` gives the bitrate for free as `size × 8 ÷ duration`.

```
skip the proxy when   height ≤ 720   AND   measured bitrate ≤ ~2 Mbps
```

On a clips:day day that removes, per clip, **one hardware transcode, one upload, and one Bunny Stream encode** (see U3). Nothing is lost: the "original" already *is* the mobile-friendly rendition — 720p, H.264, faststart.

Worth about **a third of the uploaded bytes** and the whole of the transcode time on that path, and it is a guard clause, not an architecture.

---

2. What is left after U1

With the proxy gone on that path, a clip is one PUT of ~30 MB. Twenty clips is ~600 MB. At that size the remaining levers change character:

- **Per-object latency starts to matter** relative to payload. Twenty serial PUTs with a round trip each side, plus the checks in §5, is a lot of dead air.
- **Multipart matters much less than I implied last time.** A 30 MB object does not need splitting across eight connections; Bunny themselves recommend multipart above 100 MB. The S3-zone migration drops down the list accordingly — it is for the drone-card path, if that ever uploads raw.
- **Overlapping transcode with upload (U5) also shrinks**, because on the clips:day path there is no transcode left to overlap.

So U1 does not just save bytes, it removes the justification for most of the heavier machinery. That is why it is first.

---

3. Three ways the same thing goes up twice

U2 — The cloud has no content identity for a clip

`videos` rows are deduped on `external_id` (`videos/route.ts:226`), and `external_id` is the **local** clip id, minted fresh on every import:

```
const id = `v_${Date.now()}_${Math.random().toString(36).slice(2)}`   // localStore.js:270
```

Within one browser, `saveVideo`'s `clipKey` (`date|name|size`) reuses the prior row, so the id is stable. **Across devices it is not.** The same clip imported on two laptops gets two `external_id`s, so Supabase inserts two rows and both devices upload the same footage. Nothing anywhere compares the bytes.

`clipKey` itself is also weaker than it looks in this workflow: CLAUDE.md records that the RIB camera names files `GX010041.MP4`. Name-and-size collisions across cards are a real possibility, and the only thing keeping them apart is a correctly-detected date — which is exactly the thing the timestamp forensics in `videoProbe.js` exists because we cannot always get right.

U3 — Every clip gets two Bunny Stream ladders

`putAndFetch` (`video-rendition-sync.ts:142`) is called for `kind: 'proxy'` and for `kind: 'original'`, and **both unconditionally** POST to `/api/stream/fetch` (`:236`) to queue an adaptive encode.

So each clip produces **two Stream video objects, two adaptive ladders, two sets of stored renditions** — billed twice, encoded twice. On the clips:day path the two sources are 720p and 720p, so the ladders are near-identical.

U1 removes this as a side effect. If the proxy is ever kept deliberately, only one of the two should get a ladder — the original — with the proxy serving as the immediate progressive MP4.

U4 — Three uploaders, one shared flag, and the flag arrives late

Uploader	Destination	Gate
`syncSessionToCloud` (`bunny.js:249`)	TUS → Stream	`alreadyInCloud(video)`
originals queue (`useCloudSync.js:291`)	TUS → Stream	`!v.hasOriginal`
`uploadOriginalStorageFirst` (`useBatchActions.js:260`, `UploadTab.jsx:514`)	Storage → Stream fetches	`storageObjectSize(path) === blob.size`

`alreadyInCloud` does check `hasOriginal` and `originalUploadedAt` (`bunny.js:179`), so these are not blind to each other — credit where due, and the comment above it says exactly why.

The weakness is that `hasOriginal` is **adopted from the cloud row when the day loads** (`videoBadge.ts:9`, `SSAApp.jsx:803`). Before a `loadDate` has merged the cloud row — a fresh device, a day opened offline, a clip uploaded by someone else five minutes ago — the flag is false and the gate opens. The third gate does not consult it at all; it asks Bunny for a byte count.

And note from §3 of the earlier doc, unchanged: the two TUS paths leave a clip **unwatchable for 60–120 min**, while the third makes it watchable at once. The automatic queue is still on the slow one.

---

4. The fix for all three: name media by its content

This is not a new idea here — `docs/sync-caching-architecture-research.md` §1 already specifies it, in its own words: "*Name data by the hash of its bytes… identical content is stored/transmitted once.*" The log and event files got it (`contentHash.ts`, `hashLogPayload`, `sync-manifest.json`). **Media never did.**

The fingerprint is already free

`extractVideoCreationTime` (`videoProbe.js:278`) already reads **1 MB of head and 2 MB of tail** off the drive for every clip, because `moov` can sit at either end. Those buffers are in memory at import. So:

```
fingerprint = SHA-256( size ‖ first 1 MiB ‖ last 2 MiB )
```

costs one `crypto.subtle.digest` over bytes we have already paid to read. For whole immutable media files that is ample — the research doc says so explicitly ("*our media are whole immutable files; whole-file hashing is enough*", and chunking is "*over-engineering*" for us). Two different clips agreeing on size, the first mebibyte and the last two does not happen.

What it buys

- **Storage key becomes content-addressed** — `originals/<fingerprint>.mp4` instead of `originals/<random local id>.mp4`. Two devices with the same bytes compute the same key and write the same object. Duplicate uploads become *physically impossible*, not merely detected.
- `videos` **gets a** `content_hash` **column**, unique per boat, and the POST dedupes on it ahead of `external_id`. Two devices importing one card produce one row.

- **Existence is answerable without the network** — the fingerprint is in the key, so a cached listing of the day's prefix answers for every clip at once (§5).
- **Integrity for free.** Bunny Storage takes an optional `Checksum` header (SHA-256, uppercase hex). The research doc flags the trap, which still stands: it "*returns 201 even on a bad checksum without storing the file*" — so the status code must not be trusted and the object must be verified afterwards. With the fingerprint already computed, that verification is a comparison.

What it costs

- A migration for the new column and its index.
- A back-fill decision for clips already uploaded under random keys. They do not have to move: keep reading `bunny_original_path` as stored (it is absolute and read back verbatim — the same property that made last month's tenant-scoping migration free) and only write new objects content-addressed.
- The fingerprint covers head+tail, not the whole file, so it is a strong *identity* and not a full integrity check of the middle. The `Checksum` header is the place for whole-file integrity if that is wanted.

---

5. The existence check, which is currently the expensive part

`storageObjectSize` (`bunny-storage-upload.ts:170`) answers "is this object already there?" by **listing its whole directory and scanning for the name**.

Called once per clip before each upload, so a 20-clip day performs **20 listings of the same directory**, each returning all 20 entries — quadratic work to answer a question one listing answers. Bunny's published limits allow only **5 concurrent folder listings per zone**, so this is also the operation most likely to start returning errors as a day gets bigger.

Then the comparison is `have === blob.size` — size only, no content.

U6 — One listing per prefix, cached, in front of a local manifest

Cheapest-first, which is the order §2 of the research doc sets out:

1. **Local manifest first (free).** We already write `sessions/<…>/sync-manifest.json` for log and events. Extend it to media: `{ fingerprint → { key, size, uploadedAt } }`. Most questions never reach the network.
2. **One cached directory listing per prefix per sync run.** Replaces N listings with 1. This is the documented, already-working operation and is the single change here.
3. **HEAD, if it works.** The research doc recommends HEAD generically, but Bunny's HTTP API documentation does not list it — only PUT, GET and DELETE. **Untested.** Worth one `curl -I` against a known key; if it answers, it is cheaper than a listing for the one-off case. Not something to build on until it has been tried. (The S3 endpoint certainly has `HeadObject`, but that needs the new zone.)

Do not use ETag for identity. The research doc's warning applies: a multipart ETag is not an MD5 of the file, and ETag is not guaranteed to be one even for single-part uploads. Carry our own hash.

---

6. The remaining per-object overhead

Small individually, embarrassing in aggregate once the bytes are dealt with.

	Where	Cost
U7	`photoStore.getCreds()` is **uncached** and called twice per photo (`photoStore.js:432,446,463`) — and I made that route do a Supabase auth + `users` query last month	2 server round trips per photo

	Where	Cost
U8	`auth.getUser()` **inside** the originals loop (`useCloudSync.js:245`)	0.3–0.9 s per clip
U9	Photo sync is serial across every date and photo (`photoStore.js:527`); ~6 sequential round trips each	100 photos ≈ 600 RTTs ≈ 10 min of pure latency at 80 ms
U10	Photo decode: full-res twice per photo, main thread, no `createImageBitmap` anywhere	~2 min each on the 170 MP compilations, per our own docs

U7 and U8 are one-line fixes. U9 is a bounded pool of 4–6 over independent objects. U10 is one decode at target size in a Worker.

---

7. Order

#	Change	Effect	Risk
1	**Skip the proxy when the source already satisfies it** (U1)	~⅓ of uploaded bytes, all proxy transcode time, and half the Stream encodes — on the main workflow	**Low.** A guard clause on data already in hand
2	**One cached listing per prefix** instead of per clip (U6.2)	N listings → 1; removes the quadratic term and the 5-listing ceiling	Low
3	**Cache creds; hoist** `getUser()` (U7, U8)	2 RTT per photo, ~0.5 s per clip	Trivial
4	**Background originals queue → storage-first**	kills the 60–120 min unwatchable window	Low — working call site to copy
5	**Bounded pool for photo sync** (U9)	several-fold on a day of photos	Low
6	**One decode per photo via** `createImageBitmap` **in a Worker** (U10)	170 MP compilations: ~2 min → seconds	Low–medium
7	**Content fingerprint +** `content_hash` **column + content-addressed keys** (U2/U4)	duplicate uploads become impossible rather than detected	Medium — migration, and the one real design change here
8	**Media entries in** `sync-manifest.json` (U6.1)	most existence questions stop touching the network	Medium — follows 7
9	S3 zone + multipart	only matters above ~100 MB objects; little on the clips path	High — new zone, Public Preview

1–3 are an afternoon and should be done first. They need no migration and no new concepts, and 1 alone changes the byte budget more than everything below it combined.

7 is the one that answers "no duplicate uploads" properly. Everything before it reduces duplicate *work*; only content addressing makes duplicate *bytes* impossible. I would do it after 1–6 because its value is much easier to judge once the obvious waste is gone.

---

8. What I would measure, and what I would not bother measuring

Not worth measuring: U1. The arithmetic is in the encoder flags on both sides — 720p CRF 26 in, 720p 1.2 Mbps out. There is nothing to discover.

Worth measuring, because it changes the order:

- **The real bitrate of a day's clips.** `ls -l ~/clips` plus the durations gives it in a minute. If sailing footage at CRF 26 is coming out nearer 1.5 Mbps than 3, the proxy saves almost nothing and U1 gets bigger still. If it is up near the 8M cap, the proxy is doing more than I credit it with.
- **Whether HEAD works on the zone** — one `curl -I`, settles U6.3.

- **An actual upload, timed per phase.** The upload path emits no timings at all. Copying the `_pm` pattern into the two queue drains in `useCloudSync` and into `importFiles` is a measurement-only change, and it is the thing that would tell us whether, after U1, the remaining time is bytes on the wire or our own round trips.

I would take the first two before touching anything, and add the timers as part of item 1 rather than before it.

---

9. Measured: the byte budget for one day (2 Oct 2026)

Sizes from `~/clips`, durations from `20261002.manifest.json`.

Mbps	MB	sec	clip
8.02	60.1	60	`…124245_photo`
7.83	88.1	90	`…150230_topmark`
7.82	87.9	90	`…134609_topmark`
7.73	58.0	60	`…122416_photo`
6.99	52.4	60	`…144141_photo`
6.81	68.1	80	`…123656_photo`
6.76	60.6	72	`…150559_photo`
6.32	127.9	162	`…123840_photo`
6.01	33.0	44	`…144333_photo`
5.67	103.9	147	`…120330_race-start`
5.40	80.9	120	`…131033_gate`
5.34	105.6	158	`…122642_photo`
4.70	35.2	60	`…122253_photo`
3.77	42.4	90	`…121519_topmark`
3.38	22.7	54	`…141528_gate`

15 clips · 1027 MB · 22.4 min · mean 6.10 Mbps.

What the distribution says

Four clips sit at 7.7–8.0 Mbps — **against the `--maxrate 8M` ceiling** (`compress-videos.sh:67`). With `-crf 26 -maxrate 8M` x264 does capped CRF: when CRF 26 wants more than 8 Mbps it does not get it, so those clips are **already quality-limited by the cap**, not by CRF. Water, wake and spray are high-entropy and 720p does not make them cheap.

That matters for the obvious lever. **Lowering the cap degrades exactly the clips that are already struggling** — the starts and top marks, which are the ones anyone actually studies. I would not touch `--maxrate`.

Raising CRF (26 → 28) would shrink the nine CRF-bound clips and do nothing for the four cap-bound ones. Modest, and it is a quality call that is not mine.

A more efficient codec is the textbook answer and is **closed off here**: both CLAUDE.md and `videoProbe.js:238` record that H.265 is undecodable in some browsers — "*this device's browser cannot decode the video track (often H.265/HEVC) — record in H.264*". Encoding the archive in HEVC would trade a third of the bytes for clips that are black on somebody's laptop.

The budget, and the question it forces

	MB/day	share
Originals (6.10 Mbps mean)	1027	**83%**
Proxies (1.2 Mbps)	202	17%
Total uploaded	**1229**	

Plus, from U3, **two Bunny Stream ladders per clip** — 30 adaptive encodes for 15 clips, billed and stored twice.

So: 1.2 GB per sailing day, of which five sixths is a full-quality copy that a phone will almost never play, because `url/route.ts:132` prefers the proxy or the Stream ladder for playback anyway.

U11 — make the original optional, and spend some of the saving on the proxy.

The proxy at 1.2 Mbps is a 5× cut of 6 Mbps source, at the same resolution. For high-motion water that will show. The interesting trade is not proxy-or-original, it is *what bitrate buys an acceptable review copy*:

Proxy bitrate	MB/day	vs today's 1229 MB
1.2 Mbps (today)	202	–83%
2.5 Mbps	420	–66%
3.5 Mbps	588	–52%

Even a **3.5 Mbps** proxy — nearly three times today's and a far better review copy — halves the day's upload if the original stops going up by default. Originals then upload on request, overnight, or for the clips someone marks for the archive.

This is a product decision, not a performance one: it trades "everything is in the cloud at full quality" for "everything is watchable in the cloud, full quality on request". I am not going to make that call, but the numbers say it is the only change on this list that alters the budget by more than a few percent.

---

10. Re-ranked

#	Change	Effect	Risk
1	**One cached listing per prefix** instead of one per clip (U6.2)	N listings → 1; removes the quadratic term and the 5-listing-per-zone ceiling	Low
2	**Cache creds; hoist** `getUser()` (U7, U8)	2 RTT per photo, ~0.5 s per clip	Trivial
3	**One Stream ladder per clip, not two** (U3)	halves Stream encoding and Stream storage — billed	Low
4	**Background originals queue → storage-first**	kills the 60–120 min unwatchable window	Low
5	**Bounded pool for photo sync** (U9)	several-fold on a day of photos	Low
6	**One decode per photo via** `createImageBitmap` **in a Worker** (U10)	170 MP compilations: ~2 min → seconds	Low–medium
7	**Content fingerprint +** `content_hash` **+ content-addressed keys** (U2/U4)	duplicate uploads impossible rather than detected	Medium
8	**Media in** `sync-manifest.json` (U6.1)	most existence checks stop touching the network	Medium
9	**Decide the original/proxy policy** (U11)	**–52% to –83% of all uploaded bytes**	Product decision
10	Proxy-skip guard (U1)	nothing today; insurance	Low
11	S3 zone + multipart	now justified — several clips exceed 100 MB	High

Items 1–6 are mechanical, independently shippable, and total maybe two days. They make the pipeline stop wasting round trips and Stream encodes.

Item 9 is the only one that changes the order of magnitude. Everything else is housekeeping by comparison. It wants a conversation, not a commit.

One correction to §2 above, which argued multipart had become pointless: at a mean 68 MB and a maximum of 128 MB, several clips per day are over Bunny's 100 MB multipart recommendation. It is still last — but it is no longer moot.